

Mindous.ai Build System and Deployment Pipeline

Overview

This document provides comprehensive documentation for the Build System and Deployment Pipeline implemented in Mindous.ai. The system enables autonomous agents to scaffold, build, and deploy Next.js/React applications with live preview environments.

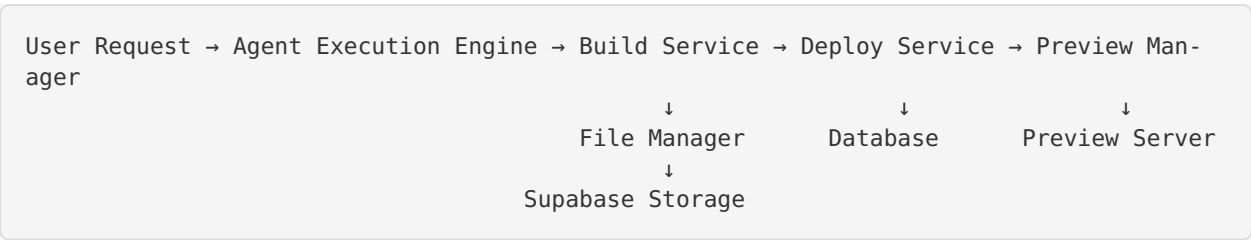
Version: 1.0.0
Date: November 17, 2025
Author: Mindous Development Team

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Core Components](#)
- 3. [Database Schema](#)
- 4. [Services](#)
- 5. [API Routes](#)
- 6. [Usage Examples](#)
- 7. [Configuration](#)
- 8. [Troubleshooting](#)

Architecture Overview

The build system consists of several interconnected services:



Key Features

- ✔ **Project Scaffolding:** Create Next.js projects from templates
- ✔ **Code Generation:** LLM-powered code file generation
- ✔ **Dependency Management:** Automatic npm package installation
- ✔ **Build Execution:** Production-ready builds with error handling
- ✔ **Preview Deployment:** Isolated preview environments with unique URLs
- ✔ **Real-time Streaming:** Live build logs and progress updates via SSE

- **✓ Artifact Management:** Storage and retrieval of build artifacts
 - **✓ Auto-cleanup:** Automatic cleanup of old builds and inactive previews
-

Core Components

1. Next.js Base Template

Location: `/lib/templates/nextjs-base/`

A minimal Next.js 14+ App Router template with:

- TypeScript support
- Tailwind CSS configuration
- ESLint setup
- Basic app structure (layout.tsx, page.tsx)
- Production-ready configuration

2. File Manager Service

Location: `/lib/services/file-manager.ts`

Responsibilities:

- Create temporary build directories (`/tmp/mindous-builds/{runId}`)
- Save and read files from temp storage
- Upload/download files to/from Supabase Storage
- Archive builds to tar.gz
- Clean up old builds

Key Methods:

```
FileManager.createBuildDir(runId: string): Promise<string>
FileManager.saveTempFile(runId: string, relativePath: string, content: string): Promise<string>
FileManager.uploadToStorage(userId: string, runId: string, filename: string, content: Buffer): Promise<string>
FileManager.cleanupBuildDir(runId: string): Promise<void>
FileManager.cleanupOldBuilds(daysOld: number): Promise<number>
```

3. Build Service

Location: `/lib/services/build-service.ts`

Responsibilities:

- Scaffold projects from templates
- Generate code files
- Update package.json dependencies
- Install npm packages
- Execute build commands
- Store build artifacts
- Publish build events

Key Methods:

```
BuildService.createProject(spec: ProjectSpec): Promise<string>
BuildService.buildProject(buildId: string, runId: string): Promise<BuildResult>
BuildService.getBuildStatus(buildId: string): Promise<any>
BuildService.getBuildLogs(buildId: string): Promise<string>
BuildService.listArtifacts(buildId: string): Promise<any[]>
```

4. Preview Manager Service

Location: /lib/services/preview-manager.ts

Responsibilities:

- Port allocation (3100-3200 range)
- Start Next.js preview servers
- Monitor server health
- Stop/restart previews
- Auto-cleanup inactive previews

Key Methods:

```
PreviewManager.startPreview(params: { previewId, buildPath, runId, buildId }):
Promise<PreviewInfo>
PreviewManager.stopPreview(previewId: string): Promise<void>
PreviewManager.checkPreviewHealth(previewId: string): Promise<boolean>
PreviewManager.cleanupInactivePreviews(hoursInactive: number): Promise<number>
```

5. Deploy Service

Location: /lib/services/deploy-service.ts

Responsibilities:

- Deploy builds to preview environments
- Generate preview URLs
- Manage deployment lifecycle
- Track deployment status

Key Methods:

```
DeployService.deployPreview(params: DeploymentParams): Promise<DeploymentInfo>
DeployService.stopDeployment(previewId: string): Promise<void>
DeployService.restartDeployment(previewId: string): Promise<DeploymentInfo>
DeployService.getDeploymentInfo(previewId: string): Promise<DeploymentInfo | null>
```

Database Schema

builds Table

Tracks build operations for generated projects.

```
CREATE TABLE builds (
  id UUID PRIMARY KEY,
  run_id UUID REFERENCES runs(id) ON DELETE CASCADE,
  execution_id UUID REFERENCES executions(id) ON DELETE CASCADE,
  user_id TEXT NOT NULL,
  project_name TEXT NOT NULL,
  project_type TEXT NOT NULL, -- nextjs, react, html, nodejs, other
  status TEXT NOT NULL, -- queued, installing, building, completed, failed, cancelled
  build_path TEXT,
  output_path TEXT,
  build_logs TEXT,
  error_message TEXT,
  started_at TIMESTAMP,
  completed_at TIMESTAMP,
  duration_ms INTEGER,
  size_bytes BIGINT,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

build_artifacts Table

Stores generated code files and assets.

```
CREATE TABLE build_artifacts (
  id UUID PRIMARY KEY,
  build_id UUID REFERENCES builds(id) ON DELETE CASCADE,
  file_path TEXT NOT NULL,
  file_type TEXT NOT NULL,
  content TEXT,
  storage_path TEXT,
  size_bytes INTEGER,
  mime_type TEXT,
  is_generated INTEGER DEFAULT 1,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);
```

preview_deployments Table

Tracks preview environments for generated apps.

```
CREATE TABLE preview_deployments (
  id UUID PRIMARY KEY,
  run_id UUID REFERENCES runs(id) ON DELETE CASCADE,
  build_id UUID REFERENCES builds(id) ON DELETE CASCADE,
  preview_url TEXT NOT NULL,
  internal_port INTEGER NOT NULL,
  status TEXT NOT NULL, -- starting, running, stopped, failed
  process_id TEXT,
  started_at TIMESTAMP,
  stopped_at TIMESTAMP,
  last_accessed_at TIMESTAMP,
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Services

Event Publishing

Build and deployment events are published in real-time via Redis:

Event Types:

- BUILD_STARTED - Build process initiated
- BUILD_LOG - Build log line
- BUILD_PROGRESS - Build progress update
- BUILD_COMPLETED - Build finished successfully
- BUILD_FAILED - Build failed
- PREVIEW_STARTING - Preview deployment starting
- PREVIEW_READY - Preview is accessible
- PREVIEW_FAILED - Preview deployment failed

Usage:

```
await EventPublisher.publishBuildEvent(runId, buildId, 'BUILD_STARTED', 'Starting build...');
await EventPublisher.publishBuildLog(runId, buildId, 'npm install output...');
```

API Routes

Build Routes

POST /api/builds/create

Create and build a new project.

Request Body:

```
{
  "runId": "uuid",
  "executionId": "uuid",
  "projectName": "my-app",
  "projectType": "nextjs",
  "files": [
    {
      "path": "app/page.tsx",
      "content": "export default function Page() { ... }"
    }
  ],
  "dependencies": {
    "axios": "^1.6.0"
  },
  "autoDeploy": true
}
```

Response:

```
{
  "success": true,
  "buildId": "uuid",
  "build": {
    "success": true,
    "buildPath": "/tmp/mindous-builds/runId",
    "outputPath": "/tmp/mindous-builds/runId/.next",
    "logs": "...",
    "durationMs": 45000,
    "status": "completed"
  },
  "preview": {
    "previewId": "uuid",
    "previewUrl": "http://localhost:3100",
    "port": 3100,
    "status": "running"
  }
}
```

GET /api/builds/[buildId]

Get build details and artifacts.

Response:

```
{
  "success": true,
  "build": {
    "id": "uuid",
    "runId": "uuid",
    "projectName": "my-app",
    "status": "completed",
    "durationMs": 45000,
    "artifactCount": 12
  },
  "artifacts": [
    {
      "id": "uuid",
      "filePath": "app/page.tsx",
      "fileType": "component",
      "sizeBytes": 1024,
      "isGenerated": true
    }
  ]
}
```

GET /api/builds/[buildId]/logs

Stream build logs as SSE or plain text.

Headers:

- Accept: text/event-stream for streaming
- Accept: text/plain for complete logs

Preview Routes

POST /api/previews/deploy

Deploy a build to preview environment.

Request Body:

```
{
  "buildId": "uuid",
  "runId": "uuid"
}
```

Response:

```
{
  "success": true,
  "preview": {
    "previewId": "uuid",
    "previewUrl": "http://localhost:3100",
    "port": 3100,
    "status": "running",
    "buildId": "uuid",
    "runId": "uuid"
  }
}
```

GET /api/previews/[previewId]

Get preview deployment info and health status.

Response:

```
{
  "success": true,
  "preview": {
    "previewId": "uuid",
    "previewUrl": "http://localhost:3100",
    "port": 3100,
    "status": "running",
    "healthy": true
  }
}
```

DELETE /api/previews/[previewId]

Stop preview deployment.

Response:

```
{
  "success": true,
  "message": "Preview stopped successfully"
}
```

POST /api/previews/[previewId] (restart)

Restart preview deployment.

Response:

```
{
  "success": true,
  "preview": {
    "previewId": "uuid",
    "previewUrl": "http://localhost:3101",
    "port": 3101,
    "status": "running"
  }
}
```

Usage Examples

Example 1: Create and Build a Simple Next.js App

```
import { BuildService } from '@lib/services/build-service';

// Create project spec
const spec = {
  runId: 'run-123',
  userId: 'user-456',
  projectName: 'my-nextjs-app',
  projectType: 'nextjs' as const,
  files: [
    {
      path: 'app/page.tsx',
      content: `
        export default function Home() {
          return <div>Hello from Mindous.ai!</div>
        }
      `
    }
  ],
  dependencies: {
    'lucide-react': '^0.263.1'
  }
};

// Create and build
const buildId = await BuildService.createProject(spec);
const result = await BuildService.buildProject(buildId, spec.runId);

if (result.success) {
  console.log('Build completed:', result.buildPath);
}
```


Example 2: Deploy Preview

```
import { DeployService } from '@lib/services/deploy-service';

// Deploy preview
const preview = await DeployService.deployPreview({
  buildId: 'build-123',
  runId: 'run-123',
  buildPath: '/tmp/mindous-builds/run-123'
});

console.log('Preview URL:', preview.previewUrl);
console.log('Preview port:', preview.port);
```

Example 3: Use Build Tool in Agent

```
import { executeBuild } from '@lib/tools/build';

// Execute build tool
const result = await executeBuild({
  runId: 'run-123',
  userId: 'user-456',
  projectName: 'my-app',
  projectType: 'nextjs',
  files: [/ * ... */],
  dependencies: { /* ... */ },
  autoDeploy: true
});

if (result.success) {
  console.log('Build ID:', result.buildId);
  console.log('Preview URL:', result.previewUrl);
}
```

Configuration

Environment Variables

```
# Database
DATABASE_URL=postgresql://...

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://...
SUPABASE_SERVICE_ROLE_KEY=...

# Redis (for event streaming)
REDIS_URL=redis://...
```

Build Configuration

Build Directory: /tmp/mindous-builds/
Preview Port Range: 3100-3200
Build Timeout: 300 seconds (5 minutes)

Max Memory: 2048 MB

Storage Bucket: mindous-artifacts

Cleanup Configuration

Old Build Cleanup: Every 24 hours

Inactive Preview Cleanup: Every 1 hour

Preview Inactivity Threshold: 1 hour

Troubleshooting

Build Fails with “Port Already in Use”

Problem: Another process is using the required port.

Solution:

1. Check active previews: `PreviewManager.getActivePreviewsList()`
2. Stop unused previews
3. Extend port range if needed

Build Fails During npm install

Problem: Network issues or package conflicts.

Solution:

1. Check network connectivity
2. Review `build_logs` in database
3. Verify package.json dependencies

Preview Not Starting

Problem: Build not completed or server startup failure.

Solution:

1. Verify build status: `BuildService.getBuildStatus(buildId)`
2. Check build logs for errors
3. Ensure build completed successfully before deploying

Cleanup Not Working

Problem: Old builds accumulating in `/tmp/mindous-builds/`.

Solution:

1. Manually trigger cleanup: `FileManager.cleanupOldBuilds(1)`
 2. Check cron job is running
 3. Verify file permissions
-

Important Notes

Localhost URLs

 **IMPORTANT:** All preview URLs use `localhost`, which refers to the **server’s localhost**, not the user’s local machine.

When sharing preview URLs:

- For development: Access from the server directly
- For production: Deploy the application to a public URL
- For testing: Use ngrok or similar tunneling service

Preview Lifecycle

Previews are automatically stopped after:

- 1 hour of inactivity (no requests)
- Manual stop via API
- Server shutdown

Storage Limits

- Max file size: 100 MB
- Storage per user: Based on plan (Free: 500MB, Pro: 10GB)
- Build retention: 24 hours in temp storage, 30 days in Supabase Storage

Monitoring

Health Checks

```
// Check preview health
const isHealthy = await PreviewManager.checkPreviewHealth(previewId);

// Get statistics
const stats = PreviewManager.getStatistics();
console.log('Active previews:', stats.activeCount);
console.log('Average uptime:', stats.averageUptime);
```

Build Metrics

```
// Get build details
const build = await BuildService.getBuildStatus(buildId);
console.log('Duration:', build.durationMs, 'ms');
console.log('Size:', build.sizeBytes, 'bytes');
```

Future Enhancements

- [] Docker container isolation for builds
 - [] CDN integration for preview assets
 - [] Kubernetes orchestration for scalability
 - [] Public preview URLs with custom domains
 - [] Build caching for faster rebuilds
 - [] Multi-region preview deployments
-

Support

For issues or questions, please:

1. Check the logs in database (`build_logs` table)
 2. Review event stream for error events
 3. Contact the development team
-

Document Version: 1.0.0

Last Updated: November 17, 2025